

COURSE REFERENCE

Java Reference

Foundations to Expert · Java 25 LTS

Generated July 2026 · anchored to the Java Language Specification & API, SE 25

How to use. This is a lookup reference, not a lesson. Browse by topic area, or find a term fast in the **alphabetical index** at the back. Each entry leads with a one-sentence essence; the detail beneath gives the mechanism, form, and a worked example.

■ CONCEPT ■ SYNTAX

Contents

Language & Runtime · 11	3
Types & Variables · 8	8
Operators, Casting & Math · 5	13
Strings & Text · 8	16

TOPIC AREA

11 entries

Language & Runtime

AT A GLANCE

Java source is compiled once by `javac` into platform-neutral bytecode, then run on any machine by the `java` launcher and its JVM. The JVM — not the compiler — produces the final machine code, and it runs any `.class` file, so other JVM languages share the same runtime. This area also covers the everyday toolchain: the edit-compile-run loop, `javap`, comments, the class/file-name rule, and how OpenJDK becomes an installable distribution like Temurin.

How Java runs CONCEPT

Java source is compiled to platform-neutral bytecode, which the JVM then executes at run time — so the same compiled program runs on any machine with a JVM.

PIPELINE



MECHANISM

`javac` turns `Foo.java` into `Foo.class` (bytecode, not machine code). `java` starts the JVM, which loads that bytecode and — via JIT compilation and interpretation — turns it into native machine code as the program runs.

WHY

The bytecode layer is the portability seam: compile once, and any operating system with a JVM can run the result. That is the whole 'write once, run anywhere' claim, made concrete.

EXAMPLE

```
javac Hello.java    # source -> Hello.class (bytecode)
java Hello          # JVM loads bytecode -> runs it
```

SOURCE **JLS SE 25** <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO `javac` · `java` · bytecode · JDK vs JRE vs JVM

javac SYNTAX

The compiler. It reads a .java source file and produces a .class bytecode file.

FORM

javac takes the full file name, extension included.

EXAMPLE

```
javac Hello.java    # requires the .java extension
# produces: Hello.class
```

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [java](#) · [How Java runs](#) · [bytecode](#)

java SYNTAX

The launcher. It starts the JVM on a compiled class and runs its main method.

FORM

java takes the class NAME only — no .java, no .class extension.

MECHANISM

The final step from bytecode to native machine code happens HERE, inside the JVM — not in javac. javac stops at bytecode.

EXAMPLE

```
java Hello    # bare class name, NO extension
# NOT: java Hello.class  (wrong)
```

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [javac](#) · [How Java runs](#) · [main method](#)

JDK vs JRE vs JVM CONCEPT

Three nested layers: the JDK builds programs, the JRE runs them, and the JVM inside it actually executes the bytecode.

THE THREE LAYERS

JDK	JRE + developer tools (javac, javap, jar). What you install to WRITE Java.
JRE	JVM + the standard class libraries. Enough to RUN a finished program.
JVM	the engine inside that actually executes the bytecode.

NESTING



USE WHEN

You need the JDK to write and compile Java. A machine that only runs finished Java programs needs just the runtime.

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [How Java runs](#) · [javac](#) · [java](#)

bytecode CONCEPT

Platform-neutral instructions stored in .class files — the JVM's 'machine code', not the CPU's.

MECHANISM

javac emits bytecode: a compact, CPU-independent instruction set the JVM understands. The JVM translates it to real machine instructions at run time via interpretation and JIT compilation. It is the JVM's instruction set, not any physical processor's.

TWO FORMS

mnemonics	human-readable ops printed by javap -c (getstatic, invokevirtual, ldc).
raw bytes	the numeric encoding of those ops stored in the .class file.

WHY IT MATTERS

Because the JVM runs any valid .class file, not Java specifically, other languages that compile to bytecode — Kotlin, Scala, Clojure, Groovy — run on the same JVM unchanged. Bytecode, not the Java language, is the portability seam.

SOURCE [JLS SE 25](https://docs.oracle.com/javase/specs/jls/se25/html/index.html) <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [How Java runs](#) · [javac](#) · [JDK vs JRE vs JVM](#)

main method SYNTAX

The fixed method signature the JVM calls to start a program — `public static void main(String[] args)`.

FORM

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

EACH KEYWORD

public	the JVM must be able to call it from outside the class
static	it runs without first creating an object of the class
void	it returns nothing to the JVM
main	the exact name the JVM looks for as the entry point
String[] args	command-line arguments, passed in as an array of strings

NOTE

Java 25 adds a simplified entry point (an instance main, no class or args boilerplate) as a preview. This course writes the classic signature above — it is the stable, universal form and what the OCP exam tests.

SOURCE **JLS SE 25** <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [java](#) · [How Java runs](#) · [class declaration](#)

edit-compile-run loop CONCEPT

After editing a .java file you must re-run javac before java, or the JVM keeps running the previous compiled version.

MECHANISM

java executes the .class file, never the .java source. Editing the source does not change the .class until javac rebuilds it. Skip the recompile and you silently run stale bytecode.

USE WHEN

Every code change: save, javac, then java. If a change seems to have no effect, an un-rerun javac is the first thing to check.

EXAMPLE

```
# edit Hello.java, then ALWAYS:  
javac Hello.java    # rebuild the .class  
java Hello          # now runs the new version
```

SOURCE **JLS SE 25** <https://docs.oracle.com/javase/specs/jls/se25/html/index.html>

SEE ALSO [javac](#) · [java](#) · [bytecode](#)

javap SYNTAX

The disassembler — prints the bytecode instructions inside a compiled .class file in readable form.

FORM

javap -c takes a class name and prints its methods as bytecode mnemonics. It reads the compiled class, not the source.

USE WHEN

Confirming what the compiler actually produced — seeing that source became bytecode, and what those instructions are.

EXAMPLE

```
javap -c Hello    # prints Hello.class bytecode
# e.g. getstatic, ldc, invokevirtual, return
```

SOURCE [JDK 25 Tool Specs](https://docs.oracle.com/en/java/javase/25/docs/specs/man/index.html) <https://docs.oracle.com/en/java/javase/25/docs/specs/man/index.html>

SEE ALSO [bytecode](#) · [javac](#)

comments SYNTAX

Notes ignored by the compiler — line comments with `//`, block comments with `/* */`, and Javadoc with `/** */`.

FORM

SYNTAX	SPANS	PURPOSE
<code>// text</code>	to end of line	a quick inline note
<code>/* text */</code>	one or more lines	a longer block note
<code>/** text */</code>	one or more lines	Javadoc — documents the class/method that follows

NOTE

All three are stripped before compilation and have no effect on the running program. Javadoc `/** */` is additionally read by the javadoc tool to generate API docs.

SOURCE [JLS §3.7](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [main method](#) · [class declaration](#)

class declaration SYNTAX

`public class Name { ... }` — and a public class must live in a file named exactly `Name.java`.

MECHANISM

A source file's public class must share the file's name, character for character. `Hello.java` must declare public class `Hello`; a mismatch is a compile error.

FORM

```
// file: Hello.java
public class Hello {
    // fields and methods
}
```

NOTE

The name is case-sensitive: `Hello` and `hello` are different. This is why `javac` takes the file name (`Hello.java`) but `java` takes the class name (`Hello`).

SOURCE [JLS §7.6](https://docs.oracle.com/javase/specs/jls/se25/html/jls-7.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-7.html>

SEE ALSO [main method](#) · [javac](#) · [java](#)

JDK distributions CONCEPT

OpenJDK is the open-source reference codebase; vendors build it into installable distributions such as Eclipse Temurin (from Adoptium).

MECHANISM

OpenJDK is the shared source. Multiple vendors compile it into ready-to-install JDKs that pass the same compatibility tests, so they behave identically for standard Java. They differ in packaging, support terms, and update cadence, not language behavior.

USE WHEN

Choosing what to install: a free, well-supported build like Temurin (Adoptium) is a standard choice. This course uses Temurin JDK 25 LTS (x64 = the 64-bit build).

NOTE

LTS (Long-Term Support) releases receive updates for years; Java 25 is an LTS. Non-LTS releases are superseded every six months.

SOURCE [Adoptium Temurin](https://adoptium.net/temurin/releases/) <https://adoptium.net/temurin/releases/>

SEE ALSO [JDK vs JRE vs JVM](#) · [How Java runs](#)

TOPIC AREA

8 entries

Types & Variables

AT A GLANCE

Java has eight primitive value types with fixed, machine-independent sizes, plus a matching wrapper class for each. A bare whole-number literal is an `int` and a bare decimal is a `double`; suffixes (`L`, `f`) and the overflow rule follow from that. Primitives are held and copied by value, which is the root of how `==` later behaves on objects versus primitives.

primitive types CONCEPT

The eight built-in value types, with fixed sizes identical on every machine, stored directly rather than as objects.

THE EIGHT PRIMITIVES

TYPE	BITS	RANGE / VALUES
byte	8	-128 to 127
short	16	-32,768 to 32,767
int	32	-2,147,483,648 to 2,147,483,647
long	64	about -9.2 to 9.2 quintillion (-2^{63} to $2^{63}-1$)
float	32	IEEE 754 single precision, about 6-7 significant digits
double	64	IEEE 754 double precision, about 15-16 significant digits
char	16	0 to 65,535 (an unsigned UTF-16 code unit)
boolean	-	true or false only

MECHANISM

A primitive variable holds the value itself, in its own fixed-size slot — not a reference to an object elsewhere. The sizes above are the same on every platform, so integer overflow and float precision behave identically everywhere.

NOTE

`int` is the default type for whole numbers and `double` for decimals; the integer types are signed two's-complement, and `char` is the one unsigned integer type.

SOURCE [JLS §4.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [int](#) · [double](#) · [wrapper classes](#) · [numeric literals](#) · [value semantics](#)

int SYNTAX

A 32-bit signed whole number — Java's default type for integer values.

RANGE

-2,147,483,648 to 2,147,483,647 (-2^{31} to $2^{31}-1$), stored as 4 bytes two's-complement.

USE WHEN

Counting, indexing, and whole-number arithmetic that stays in range. Use long when a value may exceed the int range — exceeding it wraps around silently rather than erroring.

EXAMPLE

```
int count = 42;
int sum = a + b;
```

SOURCE [JLS §4.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [primitive types](#) · [double](#) · [numeric literals](#) · [numeric promotion](#)

double SYNTAX

A 64-bit IEEE 754 floating-point number — Java's default type for decimal values.

RANGE

About 15-16 significant decimal digits. Stores a very wide range of magnitudes, but as binary fractions, so some decimals (like 0.1) are held approximately.

USE WHEN

Any calculation with fractional values where tiny rounding error is acceptable — averages, measurements, graphics. Avoid it for exact money math.

EXAMPLE

```
double g = 9.81;
double avg = total / (double) n;
```

SOURCE [JLS §4.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [primitive types](#) · [int](#) · [numeric literals](#) · [casting](#)

wrapper classes CONCEPT

An object class matching each primitive (int -> Integer, etc.), used where Java needs an object instead of a raw value.

THE EIGHT WRAPPERS

PRIMITIVE	WRAPPER
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

USE WHEN

Anywhere an object is required but a primitive is not allowed — most importantly the collections classes taught later. The wrappers also hold useful constants and helpers, e.g. Integer.MAX_VALUE and Integer.parseInt("42").

NOTE

Two of the class names differ from the primitive: int -> Integer and char -> Character. All eight live in java.lang, so no import is needed.

SOURCE [java.lang \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/package-summary.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/package-summary.html>

SEE ALSO [primitive types](#) · [int](#) · [double](#)

numeric literals SYNTAX

A bare whole number is typed int and a bare decimal is typed double; suffixes L and f change that, and underscores group digits.

DEFAULTS AND SUFFIXES

LITERAL	TYPE
100	int (bare whole number)
3.14	double (bare decimal)
100L	long (L suffix)
3.14f	float (f suffix)
1_000_000	int (underscores are ignored, for readability)

THE OVERFLOW TRAP

`long big = 10000000000;` does NOT compile. The literal is typed `int` before it is ever assigned to the `long`, and 10,000,000,000 overflows `int`'s range — so it fails at the literal itself. Write `10_000_000_000L` so it is a `long` from the start.

MECHANISM

A suffix sets the literal's type up front; it does not convert an already-typed value. Underscores may sit only between digits, never at the start, end, or next to the decimal point.

SOURCE [JLS §3.10](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [int](#) · [double](#) · [primitive types](#)

variable declaration SYNTAX

Name a typed storage slot and optionally give it a value: `type name = value;`

FORM

```
int count = 0;
String name = "Roque";
double total = 0.0;
```

NOTE

The declared type fixes what the variable can hold for its whole life. A declaration without a value (`int count;`) is legal, but the variable must be assigned before it is read.

SOURCE [JLS §4.12](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [var](#) · [primitive types](#) · [value semantics](#)

var SYNTAX

Lets the compiler infer a LOCAL variable's type from its initializer — still fully static typing, just written shorter.

MECHANISM

`var total = 0.0;` compiles exactly as `double total = 0.0;` — the compiler reads the initializer, fixes the type permanently, and `var` vanishes. The variable is not dynamically typed and cannot later hold a different type.

USE WHEN

A local variable whose type is obvious from the right-hand side. `var` requires an initializer, so the compiler has something to infer from.

CANNOT BE USED FOR

Fields, method parameters, return types, or a declaration with no initializer (`var x;` is a compile error) — none of these give the compiler an initializer to read.

SOURCE [JLS §14.4](https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html>

SEE ALSO [variable declaration](#) · [primitive types](#)

value semantics CONCEPT

Assigning or passing a primitive copies its value, so the copy and the original are fully independent afterward.

MECHANISM

A primitive variable holds its value directly. `b = a` copies `a`'s value into `b`'s own slot; changing one afterward never affects the other.

EXAMPLE

```
int a = 5;
int b = a;    // b gets a copy of 5
b = 99;      // a is still 5
```

WHY IT MATTERS

This is the contrast that makes reference types (objects) understandable later: those copy a reference to one shared object, not the object itself. The difference is exactly what `==` will hinge on.

SOURCE [JLS §4.12](https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-4.html>

SEE ALSO [primitive types](#) · [variable declaration](#)

TOPIC AREA

5 entries

Operators, Casting & Math

AT A GLANCE

Integer division truncates and `%` gives the remainder; mixing in one floating-point operand promotes the whole expression to floating point. Widening to a bigger type is automatic, narrowing to a smaller one needs an explicit cast that truncates toward zero, with a compile-time-constant exception. The `Math` methods have fixed return types worth memorizing — `pow` and `sqrt` always return `double`, `round` returns an integer type.

integer division and modulus CONCEPT

When both operands are integers, `/` discards the fractional part and `%` returns the remainder.

MECHANISM

`int / int` produces an `int`: the mathematical result is truncated toward zero, not rounded. `%` returns what is left over after that truncated division.

EXAMPLES

EXPRESSION	RESULT
<code>7 / 2</code>	3 (not 3.5 — fractional part dropped)
<code>7 % 2</code>	1 (remainder)
<code>-7 / 2</code>	-3 (truncates toward zero)

USE WHEN

This is the classic surprise: dividing two ints can never yield a decimal. Make one operand floating-point to get a real quotient (see numeric promotion).

SOURCE [JLS §15.17](https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html>

SEE ALSO [numeric promotion](#) · [casting](#)

numeric promotion CONCEPT

In a mixed-type arithmetic expression, the narrower operand is promoted so both share the wider type before the operation runs.

MECHANISM

If either operand is a floating-point type, the whole operation is done in floating point. So `9.0 / 4` promotes `4` to `4.0` and yields `2.25`, whereas `9 / 4` is pure `int` division and yields `2`.

EXAMPLES

EXPRESSION	RESULT	WHY
<code>9.0 / 4</code>	2.25	one double operand promotes the 4
<code>9 / 4</code>	2	both int, integer division
<code>5 + 2.0</code>	7.0	result is double

USE WHEN

To force a real quotient from two `int` variables, make one a `double` first: `(double) a / b`.

SOURCE [JLS §5.6](https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html>

SEE ALSO [integer division and modulus](#) · [casting](#) · [double](#)

casting CONCEPT

Widening to a larger type is automatic; narrowing to a smaller type needs an explicit (type) cast and can lose data.

WIDENING (AUTOMATIC)

A smaller type flows into a larger one with no cast, because no information is lost: `int` -> `long` -> `double`, for example. `int x = 5; double d = x;` is fine.

WIDENING PATH

`byte` → `short` → `int` → `long` → `float` → `double`

NARROWING (EXPLICIT)

Going to a smaller type requires a cast and may lose data. A floating-to-integer cast truncates toward zero — it does not round: `(int) 9.99` is 9 and `(int) -9.99` is -9.

EXAMPLE

```
double d = 9.99;
int n = (int) d;    // 9 (truncated, not rounded)
long big = 5;      // widening, no cast needed
```

SOURCE [JLS §5](https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html>

SEE ALSO [final-constant narrowing](#) · [numeric promotion](#) · [primitive types](#)

final-constant narrowing CONCEPT

A final variable holding a compile-time constant that fits the target can be assigned to a narrower type **WITHOUT** a cast.

MECHANISM

When the right-hand side is a final variable whose constant value is known at compile time and fits the smaller type's range, the compiler allows the assignment with no cast. Drop `final` (or exceed the range) and it becomes a normal narrowing that fails to compile.

EXAMPLE

```
final int x = 10;
byte b = x;    // compiles: 10 is constant and fits byte

int y = 10;
byte c = y;    // ERROR: y is not final, needs (byte) y
```

USE WHEN

Mostly an exam and edge-case detail — it explains why an apparently identical assignment compiles in one place and not another.

SOURCE [JLS §5.2](https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-5.html>

SEE ALSO [casting](#) · [primitive types](#)

Math methods CONCEPT

The `java.lang.Math` helpers have fixed return types worth memorizing — notably `pow/sqrt` always return `double` and `round` returns an integer type.

RETURN TYPES (VERIFIED ON JDK 21)

CALL	RESULT	RETURN TYPE
<code>Math.pow(2, 10)</code>	1024.0	double (always)
<code>Math.sqrt(16)</code>	4.0	double (always)
<code>Math.round(9.5)</code>	10	long (from a double arg)
<code>Math.round(9.5f)</code>	10	int (from a float arg)
<code>Math.abs(-7)</code>	7	matches the argument type
<code>Math.max(3.5, 7)</code>	7.0	double (7 promoted)

NOTE

`pow` and `sqrt` return `double` even when the result is a whole number, so `Math.pow(2,10)` is 1024.0, not 1024. `round` is the one that hands back an integer type: `long` for a double argument, `int` for a float argument.

SOURCE [java.lang.Math \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Math.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Math.html>

SEE ALSO [numeric promotion](#) · [casting](#) · [double](#)

TOPIC AREA

8 entries

Strings & Text

AT A GLANCE

A `String` is an immutable object — every method that seems to change it actually returns a new one. That single fact explains the literal pool, why `==` is the classic trap for anyone coming from Python, and why `StringBuilder` exists. Text blocks add readable multi-line literals, with precise, easy-to-miss whitespace rules.

String CONCEPT

An immutable sequence of characters (UTF-16) — a reference type, not a primitive, with special literal and + syntax.

MECHANISM

A String is an object holding a fixed character sequence. The language gives it two conveniences no other class gets: double-quote literals ("hi") and the + concatenation operator.

EXAMPLE

```
String s = "hello";
String greeting = s + ", world";
```

SOURCE [java.lang.String \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>

SEE ALSO [String immutability](#) · [== vs .equals\(\)](#) · [StringBuilder](#) · [string pool](#)

String immutability CONCEPT

Once a String exists, its contents can never change — every 'modifying' method returns a brand-new String and leaves the original untouched.

MECHANISM

Methods like `toUpperCase()`, `trim()`, and `replace()` do not edit the String in place. They compute and return a new String object; the original is unchanged.

PROOF

Verified empirically in-lesson: calling `s.toUpperCase()` and then re-checking `s` showed `s` was untouched — the uppercased text only exists in the returned value.

WHY IT MATTERS

Immutability is what makes the literal pool and thread-safety possible — a shared String can never be altered out from under another user of it.

EXAMPLE

```
String s = "hi";
s.toUpperCase();           // result thrown away!
String big = s.toUpperCase(); // correct: capture it
```

SOURCE [java.lang.String \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>

SEE ALSO [string pool](#) · [StringBuilder](#) · [String](#)

string pool CONCEPT

Identical String literals share one single object in a cache called the pool — safe only because Strings can never change.

MECHANISM

When the same literal text appears more than once, Java reuses one pooled object rather than making duplicates. Because Strings are immutable, sharing is safe — no user of the shared object can mutate it.

CONSEQUENCE

Two identical literals are literally the same object, so `==` between them happens to return `true`. This is an implementation detail — never rely on it for comparison.

SOURCE [JLS §3.10.5](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [String immutability](#) · [new String\(\)](#) · `==` vs `.equals()`

new String() SYNTAX

Explicitly constructs a fresh String object that deliberately does NOT come from the pool — even if an identical literal already exists.

MECHANISM

`new String("a")` forces a distinct object. So `new String("a") == "a"` is `false` (different objects), even though their contents are equal.

USE WHEN

Almost never in normal code. Its main value is illustrating the pool: it is the deliberate way to opt out of literal sharing.

EXAMPLE

```
String a = "a";
String b = new String("a");
a == b;           // false – different objects
a.equals(b);     // true  – same contents
```

SOURCE [java.lang.String \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>

SEE ALSO [string pool](#) · `==` vs `.equals()` · [String immutability](#)

== vs .equals() CONCEPT

`==` asks 'are these the same object?' (reference identity); `.equals()` asks 'do these have the same contents?' — for Strings, always use `.equals()`.

MECHANISM

`==` compares references: are both variables pointing at the exact same object? `.equals()` (as `String` overrides it) compares the actual character sequences.

THE PYTHON TRAP

In Python, `==` compares values. In Java, `==` on any object compares identity, not value. Carrying the Python habit over is the classic source of `String` bugs.

EXAMPLE

```
String a = new String("hi");
String b = new String("hi");
a == b;           // false – two objects
a.equals(b);      // true  – same text
```

SOURCE [JLS §15.21](https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html>

SEE ALSO [string pool](#) · [new String\(\)](#) · [String immutability](#)

StringBuilder CONCEPT

A mutable character buffer — the right tool for building a `String` piece by piece, because it edits one buffer in place instead of creating a new object per step.

MECHANISM

`StringBuilder` holds a growable internal buffer (initial capacity 16). `append()` and `insert()` modify that one buffer; you call `toString()` at the end to get the finished `String`.

USE WHEN

Assembling text in a loop, or from many pieces. For a couple of + operations, plain `String` is fine and clearer.

EXAMPLE

```
StringBuilder sb = new StringBuilder();
for (String w : words) sb.append(w).append(' ');
String line = sb.toString();
```

SOURCE [java.lang.StringBuilder \(API\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/StringBuilder.html) <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/StringBuilder.html>

SEE ALSO [String immutability](#) · [String](#)

text blocks SYNTAX

Multi-line String literals delimited by triple quotes — readable JSON, HTML, or SQL without escaped newlines.

FORM

Open with `"""` followed immediately by a newline; write the content on following lines; close with `"""`. The result is an ordinary String.

EXAMPLE

```
String json = """
    {
        "name": "Roque"
    }
    """;
```

SOURCE [JLS §3.10.6](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [text block whitespace](#) · [String](#)

text block whitespace SYNTAX

Text blocks strip the whitespace you didn't mean and keep the whitespace you did — via a few precise rules worth memorizing.

INCIDENTAL LEADING WHITESPACE

The common indentation shared by all lines (measured against the least-indented line and the closing `"""` position) is stripped automatically, so you can indent the block to match your code without that indentation appearing in the string.

TRAILING WHITESPACE

Whitespace at the end of each line is removed automatically.

KEEP A TRAILING SPACE: `\s`

End a line with the `\s` escape to preserve exactly one trailing space that would otherwise be stripped.

JOIN LINES: TRAILING `\`

End a line with a backslash to suppress its newline — the next line continues the same logical line.

SOURCE [JLS §3.10.6](https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html) <https://docs.oracle.com/javase/specs/jls/se25/html/jls-3.html>

SEE ALSO [text blocks](#) · [String](#)

Index

Every term, alphabetical, with its page.

== vs .equals()	19	main method	5
bytecode	5	Math methods	16
casting	14	new String()	18
class declaration	8	numeric literals	11
comments	7	numeric promotion	14
double	10	primitive types	9
edit-compile-run loop	6	String	17
final-constant narrowing	15	String immutability	17
How Java runs	3	string pool	18
int	10	StringBuilder	19
integer division and modulus	14	text block whitespace	20
java	4	text blocks	20
javac	4	value semantics	13
javap	7	var	12
JDK distributions	8	variable declaration	12
JDK vs JRE vs JVM	5	wrapper classes	11